

Práctica de Bucles, Funciones y Estructuras de Datos

Bucles for / while, Funciones, Listas y Tuplas · Python

[Bucle for](#)[Bucle while](#)[Funciones](#)[Listas](#)[Métodos de listas](#)[Listas anidadas](#)[Tuplas](#)**SISTEMA**

Python 3 · VS Code

ARCHIVO

`practica_clase2_3.py`

TIEMPO ESTIMADO

100 – 130 minutos, individual

REQUISITO PREVIO

Tema 1, Clase 2: Bucles y Funciones · Tema 2, Clase 3: Listas y Tuplas (Estructuras de Datos)

OBJETIVO

Escribir tú mismo cada bloque de código, con la solución disponible para autoevaluarte, y cerrar con un desafío sin solución que combina los 7 conceptos en un caso nuevo

ENTREGA

Archivo `practica_clase2_3.py` corriendo sin errores, de principio a fin, incluyendo el Desafío Final

Cómo usar esta práctica

Esta práctica es individual. Vas a aplicar los mismos 7 conceptos que aprendiste en la Clase 2 (bucles `for` y `while`, funciones) y en la Clase 3 (listas, sus métodos, listas anidadas y tuplas), pero con situaciones distintas a las que usó tu instructor en clase. Además de cada instrucción, en algunos pasos vas a encontrar preguntas de razonamiento que no se responden con código, sino pensando antes de escribir.

CÓMO TRABAJAR CADA PASO

Recordatorio breve: 2 o 3 líneas que refrescan el concepto, no una clase completa otra vez.

Instrucción: la situación específica que tienes que resolver con código.

Solución: inténtalo primero tú mismo, en tu archivo `practica_clase2_3.py`, antes de mirarla. Está ahí para que confirmes que tu código está bien, no para copiarla de entrada. Tápala con la mano o con otra ventana si notas que tus ojos se van directo a ella.

Pregunta de razonamiento (cuando aplica): contéstala con una frase corta, antes de correr cualquier código, para confirmar que entendiste el concepto y no solo copiaste un patrón.

EL DESAFÍO FINAL ES DISTINTO

Después de los 7 pasos, esta práctica cierra con un Desafío Final que combina bucles, funciones, listas y tuplas en un caso nuevo, más complejo que los anteriores, y sin código de solución. Ahí sí vas a tener que resolverlo sin un ejemplo resuelto al lado, apoyándote solo en 2 casos de prueba, con datos de entrada distintos, para confirmar que tu resultado es correcto.

NOTA SOBRE EL ARCHIVO

Todo el código de esta práctica va en un solo archivo: `practica_clase2_3.py`. Puedes ir agregando cada paso debajo del anterior, o comentando los pasos anteriores con `#` mientras trabajas en uno nuevo, para que la salida de la terminal no se mezcle.

SOBRE EL RITMO

Son 7 conceptos en una sola práctica, así que está perfectamente bien dividirla en 2 sesiones: por ejemplo Pasos 1 a 3 (bucles y funciones) un día, y Pasos 4 a 7 (listas y tuplas) más el Desafío Final otro día. Entender bien cada paso vale más que terminarla de una sola sentada.

Bucle for

Recordatorio: `for` repite un bloque de código una vez por cada elemento de algo iterable (una lista, un rango). Es ideal cuando ya saben sobre qué van a recorrer.

 **Conexión con IA:** cuando un modelo entrena, recorre miles de ejemplos de un dataset uno por uno, con esta misma lógica de un `for`.

INSTRUCCIÓN, PARTE A

Con la lista `horas_estudio = [2, 3, 1, 4, 2, 5, 3]` (horas estudiadas cada día de la semana), calcula e imprime el total de horas estudiadas y el promedio diario, redondeado a 1 decimal.

No copies el ejemplo de la clase con edades y un contador de mayores de 30. Aquí son horas de estudio, y el primer cálculo es una suma total, no un conteo.

SOLUCIÓN, PARTE A

```
horas_estudio = [2, 3, 1, 4, 2, 5, 3]

total = 0
for h in horas_estudio:
    total += h

promedio = total / len(horas_estudio)
print("Total de horas:", total)
print("Promedio diario:", round(promedio, 1))
```

SALIDA ESPERADA

Total de horas: 20
Promedio diario: 2.9

INSTRUCCIÓN, PARTE B

Usando la misma lista, cuenta cuántos días estudiaste 3 horas o más, con una variable contadora.

SOLUCIÓN, PARTE B

```
contador = 0
for h in horas_estudio:
    if h >= 3:
        contador += 1

print("Días con 3 horas o más:", contador)
```

SALIDA ESPERADA

Días con 3 horas o más: 4

PREGUNTA DE RAZONAMIENTO

Sin correr ningún código todavía: si la condición pidiera 4 horas o más en vez de 3, ¿cuántos días cumplirían? Escribe tu respuesta antes de probarlo.

RESPUESTA

Con `h >= 4` : solo los días con 4 y 5 horas cumplen, así que el contador daría 2. Pueden confirmarlo cambiando el `3` por un `4` en su código y corriéndolo de nuevo.

Bucle while

Recordatorio: `while` repite un bloque mientras una condición siga siendo verdadera, sin saber de antemano cuántas veces va a repetirse. Necesita algo adentro que eventualmente vuelva la condición falsa, o el bucle nunca termina.

 **Conexión con IA:** esta misma lógica de repetir hasta cumplir una condición aparece en muchos procesos de entrenamiento y procesamiento de datos en IA, aunque cada framework la implemente a su manera.

INSTRUCCIÓN

Simula un plan de ahorro: empiezas con \$0 y ahorras \$45 cada semana. Usa `while` para saber cuántas semanas completas necesitas para alcanzar al menos \$300, e imprime el número de semanas y el ahorro final acumulado.

Es el mismo tipo de repetición condicional que vieron con la cuenta regresiva en la Clase 2, pero aquí el bucle avanza hacia adelante (sumando), no hacia atrás (restando), y no saben de antemano cuántas vueltas va a necesitar.

SOLUCIÓN

```
ahorro = 0
semanas = 0

while ahorro < 300:
    ahorro += 45
    semanas += 1

print("Semanas necesarias:", semanas)
print("Ahorro final:", ahorro)
```

SALIDA ESPERADA

Semanas necesarias: 7
Ahorro final: 315

PREGUNTA DE RAZONAMIENTO

Sin correr el código: si ahorraras \$60 por semana en vez de \$45, ¿cuántas semanas necesitarías para llegar a los \$300?

RESPUESTA

\$300 dividido entre \$60 da exactamente 5, así que en la semana 5 el ahorro llega a \$300 justo ($5 \times 60 = 300$) y el bucle se detiene ahí: 5 semanas, ahorro final \$300. Pueden confirmarlo cambiando `ahorro += 45` por `ahorro += 60` en su código.

Funciones

Recordatorio: una función se define con `def`, recibe parámetros entre paréntesis, y `return` es lo que devuelve como resultado para que quien la llamó pueda usarlo.

 **Conexión con IA:** en un pipeline de IA, cada paso (limpiar datos, calcular una métrica, hacer una predicción) suele vivir en su propia función reutilizable, igual que aquí.

INSTRUCCIÓN, PARTE A

Escribe una función `calcular_costo_envio(peso, distancia)` que calcule el costo de un envío con la fórmula: $\text{peso} * 0.5 + \text{distancia} * 0.1$. Llámala con un paquete de 12 kg que viaja 80 km, e imprime el resultado redondeado a 2 decimales.

No copien el ejemplo del IMC de la clase. Aquí la fórmula es distinta y el dominio es un cálculo de envíos, no de salud.

SOLUCIÓN, PARTE A

```
def calcular_costo_envio(peso, distancia):
    costo = peso * 0.5 + distancia * 0.1
    return costo

resultado = calcular_costo_envio(12, 80)
print(f"Costo: {round(resultado, 2)}")
```

SALIDA ESPERADA

Costo: 14.0

INSTRUCCIÓN, PARTE B

Escribe una segunda función `clasificar_envio(costo)` que reciba un costo y devuelva "Económico" si es menor a 10, "Estándar" si es de 10 a 24.99, y "Premium" si es 25 o más. Llámala con el resultado de la parte A.

SOLUCIÓN, PARTE B

```
def clasificar_envio(costo):
    if costo < 10:
        return "Económico"
    elif costo < 25:
        return "Estándar"
    else:
        return "Premium"

print(clasificar_envio(resultado))
```

SALIDA ESPERADA

Estándar

PREGUNTA DE RAZONAMIENTO

Sin correr el código: ¿qué categoría de envío devolvería `clasificar_envio(30)` ?

RESPUESTA

"Premium", porque 30 no es menor a 10 ni menor a 25, así que cae en el `else` final.

Listas: creación, acceso y slicing

Recordatorio: una lista guarda varios valores en un orden fijo. Los índices empiezan en 0, los índices negativos cuentan desde el final, y el slicing `lista[a:b]` devuelve un rango sin incluir la posición `b`.

 **Conexión con IA:** el slicing es una herramienta que vas a usar todo el tiempo al preparar datasets, por ejemplo para quedarte con distintos subconjuntos de tus datos (como entrenamiento y prueba).

INSTRUCCIÓN, PARTE A

Con la lista `peliculas` de 6 títulos, imprime la lista completa, la primera película, la última (con índice negativo), las primeras 3 (con slicing), y cuántas películas hay en total.

No copien el ejemplo de frutas de la clase. Aquí es una lista de películas, con sus propias 6 posiciones.

SOLUCIÓN, PARTE A

```
peliculas = ["Coco", "Interstellar", "Matrix", "Toy Story", "Origen", "Encanto"]

print(peliculas)
print(peliculas[0])
print(peliculas[-1])
print(peliculas[:3])
print(len(peliculas))
```

SALIDA ESPERADA

```
['Coco', 'Interstellar', 'Matrix', 'Toy Story', 'Origen', 'Encanto']
Coco
Encanto
['Coco', 'Interstellar', 'Matrix']
6
```

INSTRUCCIÓN, PARTE B

Usando slicing con paso, imprime las películas en las posiciones pares (0, 2, 4). Después imprime las películas desde la posición 1 hasta la 3 (sin incluir la 4).

SOLUCIÓN, PARTE B

```
print(peliculas[::2])
print(peliculas[1:4])
```

SALIDA ESPERADA

```
['Coco', 'Matrix', 'Origen']
['Interstellar', 'Matrix', 'Toy Story']
```

PREGUNTA DE RAZONAMIENTO

Sin correr el código: ¿qué devolvería `peliculas[2:]` ?

RESPUESTA

Dejar el segundo número vacío significa "hasta el final", así que devuelve desde la posición 2 en adelante: `['Matrix', 'Toy Story', 'Origen', 'Encanto']`.

Métodos de listas

Recordatorio: `.append()` agrega al final, `.insert(pos, valor)` inserta en una posición específica, `.remove(valor)` quita la primera aparición, `in` pregunta si un valor existe, y `.sort()` ordena la lista modificándola directamente.

 **Conexión con IA:** antes de entrenar un modelo es común limpiar una lista de datos agregando, quitando o revisando valores, con estos mismos métodos.

INSTRUCCIÓN

Empieza con `carrito = ["pantalon", "camisa"]`. Agrega "zapatos" al final, agrega "cinturon" en la posición 1, quita "pantalon", verifica si "camisa" sigue en el carrito, y por último ordena el carrito alfabéticamente. Imprime el carrito después de cada paso.

No copien el ejemplo de frutas de la clase. Aquí es un carrito de compras, con sus propios 3 movimientos.

SOLUCIÓN

```
carrito = ["pantalon", "camisa"]

carrito.append("zapatos")
print(carrito)

carrito.insert(1, "cinturon")
print(carrito)

carrito.remove("pantalon")
print(carrito)

print("camisa" in carrito)

carrito.sort()
print(carrito)
```

SALIDA ESPERADA

```
['pantalon', 'camisa', 'zapatos']
['pantalon', 'cinturon', 'camisa', 'zapatos']
['cinturon', 'camisa', 'zapatos']
True
['camisa', 'cinturon', 'zapatos']
```

PREGUNTA DE RAZONAMIENTO

Sin correr el código: si después de ordenar escribieran `carrito = carrito.sort()`, ¿qué quedaría guardado en `carrito`?

RESPUESTA

None. `.sort()` ordena la lista original pero no devuelve nada útil directamente, así que reasignar el resultado a la variable les haría perder su lista. Es uno de los errores más comunes al aprender listas.

Si alguna vez necesitan conservar la lista original y quedarse además con una copia ordenada, existe `sorted(lista)`: a diferencia de `.sort()`, esta sí devuelve una lista nueva y no toca la original.

Listas anidadas

Recordatorio: una lista puede contener otras listas. Se accede encadenando corchetes (`lista[filas][columna]`), y un `for` sobre la lista externa entrega, en cada vuelta, una de las listas internas completas.

💡 **Conexión con IA:** una lista anidada es, en esencia, una matriz: la estructura que usan el álgebra lineal y las redes neuronales para representar datos e imágenes.

INSTRUCCIÓN, PARTE A

Con el mapa de asientos `asientos` (una cuadrícula 3×3 de "L" libre / "X" ocupado), imprime la primera fila completa, el asiento en la fila 1 columna 2, y luego recorre todas las filas con un `for` para imprimir cada una.

No copien el tablero de X y O de la clase. Aquí es un mapa de asientos, y usamos "X" para "ocupado" en vez de "O" para no confundirlo con el número cero al leerlo.

SOLUCIÓN, PARTE A

```
asientos = [
    ["L", "X", "L"],
    ["X", "X", "L"],
    ["L", "L", "X"],
]

print(asientos[0])
print(asientos[1][2])

for fila in asientos:
    print(fila)
```

SALIDA ESPERADA

```
['L', 'X', 'L']
L
['L', 'X', 'L']
['X', 'X', 'L']
['L', 'L', 'X']
```

INSTRUCCIÓN, PARTE B

Con `ventas_vendedores` (nombre + 3 ventas por vendedor), calcula e imprime, para cada vendedor, su nombre y el total de sus ventas.

Antes de escribir nada: ¿qué contiene `vendedor` en la primera vuelta del `for`, y qué te devuelve `vendedor[1:]` ?

Listas anidadas

SOLUCIÓN, PARTE B

```
ventas_vendedores = [  
    ["Marta", 320, 450, 275],  
    ["Julio", 180, 200, 210],  
]  
  
for vendedor in ventas_vendedores:  
    nombre = vendedor[0]  
    ventas = vendedor[1:]  
    total = sum(ventas)  
    print(f"{nombre}: total de ventas ${total}")
```

SALIDA ESPERADA

Marta: total de ventas \$1045
Julio: total de ventas \$590

PREGUNTA DE RAZONAMIENTO

Sin correr el código: ¿qué expresión usarías para acceder directamente al segundo valor de ventas de Julio (el 200), sin recorrer la lista con un `for` ?

RESPUESTA

`ventas_vendedores[1][2]` . El `[1]` entra a la lista de Julio, y el `[2]` toma su tercer elemento, que es el segundo valor de ventas (200), porque el primer elemento `[0]` es el nombre.

Tuplas

Recordatorio: una tupla se escribe con paréntesis en vez de corchetes, se accede igual que una lista, pero no se puede modificar después de creada (inmutabilidad).

 **Conexión con IA:** una tupla es útil para representar valores que conceptualmente deben mantenerse juntos y no modificarse; en IA y ciencia de datos vas a encontrar estructuras inmutables como esta en varios contextos.

INSTRUCCIÓN, PARTE A

Crea una tupla `color = (120, 200, 50)` que representa un color en RGB. Imprímela completa, cada canal por separado, y su `type()`.

No copien la coordenada geográfica de la clase. Aquí la tupla representa un color, otro ejemplo clásico de valor que no debería cambiar una vez definido.

SOLUCIÓN, PARTE A

```
color = (120, 200, 50)

print(color)
print(color[0], color[1], color[2])
print(type(color))
```

SALIDA ESPERADA

```
(120, 200, 50)
120 200 50
<class 'tuple'>
```

AHORA INTENTA MODIFICAR UN CANAL DIRECTAMENTE

```
color[0] = 255
```

ERROR

```
TypeError: 'tuple' object does not support item assignment
```

Tuplas

INSTRUCCIÓN, PARTE B

Usa una función `calcular_brillo(c)` que devuelva el promedio de los 3 canales de un color, para comparar `color_a = (120, 200, 50)` contra `color_b = (10, 10, 10)` e imprimir cuál es más brillante.

SOLUCIÓN, PARTE B

```
color_a = (120, 200, 50)
color_b = (10, 10, 10)

def calcular_brillo(c):
    return sum(c) / 3

brillo_a = calcular_brillo(color_a)
brillo_b = calcular_brillo(color_b)
print(f"Brillo A: {round(brillo_a, 2)}")
print(f"Brillo B: {round(brillo_b, 2)}")

if brillo_a > brillo_b:
    print("El color A es más brillante")
else:
    print("El color B es más brillante")
```

SALIDA ESPERADA

```
Brillo A: 123.33
Brillo B: 10.0
El color A es más brillante
```

PREGUNTA DE RAZONAMIENTO

¿Por qué tiene sentido guardar un color RGB en una tupla en vez de en una lista?

RESPUESTA

Porque los 3 valores de un color representan un dato fijo: una vez que definen qué color es, esos 3 números no deberían cambiar por separado. La inmutabilidad de la tupla comunica esa intención sin necesitar un comentario.

Torneo de trivia por equipos

Sin código de ejemplo esta vez, y sin solución. Los 7 pasos anteriores te dieron el patrón resuelto para cada concepto por separado. Este desafío no repite ningún ejemplo ya visto: combínalos tú mismo, en un programa que reciba una lista de equipos y la procese de principio a fin.

INSTRUCCIÓN

Vas a recibir una lista de equipos, donde cada equipo es una lista con el nombre seguido de 3 puntajes de ronda, por ejemplo: `["Los Rayos", 15, 20, 18]`. Con esa lista de equipos, tu programa debe:

1. Con un `for`, calcular el total de puntos de cada equipo (la suma de sus 3 rondas) e imprimir, para cada uno, su nombre, su total, y su clasificación.
2. Escribir una función `clasificar_equipo(total)` que devuelva "Campeón" si el total es 60 o más, "Finalista" si es de 40 a 59, y "Participante" si es menor a 40.
3. Encontrar cuál equipo tiene el total más alto.
4. Con un `while`, simular una ronda bonus para ese equipo: cada vuelta suma 5 puntos extra hasta que su total llegue o supere los 70 puntos. Contar cuántas rondas bonus tomó, e imprimir el nombre del equipo, las rondas necesarias, y el total final tras la ronda bonus.
5. Guardar el resultado del campeonato en una tupla `(nombre, clasificación)` con el nombre del equipo que tuvo el total más alto y su clasificación, e imprimirla.

Si el equipo con más puntos ya tiene 70 o más desde el inicio, el `while` de la ronda bonus simplemente no debería entrar ninguna vez: 0 rondas es una respuesta válida. Piensa cómo tu condición ya cubre ese caso sin necesitar código extra.

CÓMO AUTOEVALUARTE, SIN VER LA SOLUCIÓN

Aquí no hay código de referencia. En su lugar, tienes 2 casos de prueba con la lista de entrada y el resultado exacto que deberían dar. Corre tu programa con estos mismos valores, y compara tu salida contra la de aquí. Si coincide en ambos casos, tu lógica está correcta.

CASO DE PRUEBA 1

```
equipos = [
    ["Los Rayos", 15, 20, 18],
    ["Estrellas FC", 22, 19, 25],
    ["Team Nova", 10, 12, 8],
]
```

Los Rayos: total 53, Finalista
 Estrellas FC: total 66, Campeón
 Team Nova: total 30, Participante
 Equipo con más puntos: Estrellas FC
 Rondas bonus necesarias para llegar a 70: 1
 Total final tras ronda bonus: 71
 Resultado final: ('Estrellas FC', 'Campeón')

CASO DE PRUEBA 2

```
equipos = [
    ["Halcones", 12, 14, 10],
    ["Titanes", 30, 28, 15],
    ["Fenix", 18, 16, 20],
]
```

Halcones: total 36, Participante
 Titanes: total 73, Campeón
 Fenix: total 54, Finalista
 Equipo con más puntos: Titanes
 Rondas bonus necesarias para llegar a 70: 0
 Total final tras ronda bonus: 73
 Resultado final: ('Titanes', 'Campeón')

SI TU RESULTADO NO COINCIDE

Imprime cada variable intermedia para ver dónde se desvía tu resultado. Empieza por el Caso 2: si suma rondas bonus de más, revisa la condición del `while`.

Autoevaluación

Marca cada punto solo si lo escribiste tú mismo, sin mirar la solución antes de intentarlo.

- ☐ Escribí el Paso 1 (bucle for) sin copiar el ejemplo de la clase

- ☐ Escribí el Paso 2 (bucle while) y entiendo por qué necesita algo que lo detenga

- ☐ Escribí las 2 funciones del Paso 3 y entiendo qué hace return

- ☐ Practiqué índices y slicing del Paso 4 sin ver el ejemplo de la clase

- ☐ Usé los métodos de listas del Paso 5, y entiendo qué devuelve sort()

- ☐ Recorrí la lista anidada del Paso 6 y calculé los totales correctamente

- ☐ Entiendo cuándo conviene usar una tupla en vez de una lista (Paso 7)

- ☐ Mi Desafío Final coincide con los 2 casos de prueba, sin haber visto una solución

- ☐ Mi archivo practica_clase2_3.py corre de principio a fin sin ningún error

SI TE TRABASTE EN ALGÚN PASO

Repasa tus apuntes y los ejemplos de la Clase 2 (bucles y funciones) o de la Clase 3 (listas y tuplas), y vuelve a leer el Paso correspondiente de esta misma práctica antes de intentarlo de nuevo. Si sigues sin entenderlo, preguntale a tu instructor antes de la próxima clase: estos 7 conceptos se siguen usando en cada clase que sigue, especialmente cuando lleguen a diccionarios y conjuntos, no son un tema aislado.